UNIVERSITÉ DE REIMS CHAMPAGNE-ARDENNE

Master 1 Réseaux et Télécommunications · 2026-2027

CineVault

Enterprise

Conception, réalisation et validation d'une vidéothèque répartie

RT0705 · Python, Flask, Jinja et stockage JSON
Docker Compose puis Docker Swarm sur trois VM

Mathieu Kuhnapfel

État technique du 9 novembre 2026, 16 h 04, heure de Paris

Rapport technique - preuves et limites documentées
Ce document distingue exigences académiques, extensions, observations et limites. Les preuves d'administration privées et les secrets sont exclus de la publication.

Résumé

CineVault est une application collaborative de gestion de vidéothèque. Le navigateur utilise un serveur Flask/Jinja, lequel appelle une API Flask distincte par HTTP/JSON. L'API est seule propriétaire du document JSON métier. Les films, exemplaires, collections privées, prêts et réservations sont reliés par des identifiants stables ; les transactions sont protégées entre processus et les écritures remplacent atomiquement le fichier validé.

Le socle pédagogique a été exécuté sous Docker Compose avec code hôte monté et données séparées, puis étendu à trois VM Ubuntu 24 : présentation, API et manager Docker Swarm dédié. Les images de registre public sont reliées à des CI Windows/Ubuntu réussies et déployées par digest. Au jalon décrit, les suites comportent173 tests API et159 tests web sur chacune de ces plateformes ; la dernière mise à jour distribuée a passé25 contrôles avec conservation des données et sessions. Ces nombres décrivent des campagnes précises, et non un taux universel de conformité ou de sécurité.

Le rapport explique les choix, les traitements à réception des messages, les contrôles réellement exécutés et leurs limites. Le laboratoire demeure sans haute disponibilité ni terminaison HTTPS publique. Le boot autonome de deux VM et la reprise après perte de disque ne sont pas démontrés. Le corpus des exemples, les contrôles et leurs limites accompagnent les sources ; les matrices et les pièces de clôture précisent leur portée.

Mots-clés : application répartie, REST, Flask, Jinja, JSON, verrou interprocessus, Swarm, provenance d'image.

Sommaire

1. Introduction 4
2. Analyse du sujet et des cours 4
3. Cahier des charges et critères de réussite 4
4. Méthodologie et conservation de l'existant 4
5. Acteurs, responsabilités et charge 4
6. Personas et parcours prioritaires 5
7. Cas d'utilisation et autorisations 5
8. Modèle fonctionnel 5
9. Modèle de données et exemples 5
10. Modèles d'exécution et messages 6
11. Architecture à trois tiers 6
12. Inventaire du frontend et des scripts 7
13. Spécification REST 7
14. Infrastructure RemoteLabz 8
15. Docker Compose et volume partagé 9
16. Docker Swarm et distribution des images 9
17. Développement du frontend et robustesse 10
18. Développement du backend 10
19. Authentification et autorisations 10
20. Collections et exemplaires 10
21. Emprunts et réservations 11
22. Persistance JSON et reprise 11
23. Extensions externes et calculs 11
24. Stratégie de tests et résultats 11
25. Sécurité et limites du contrôle 12
26. Résultats observés 12
27. Problèmes rencontrés et corrections 12
28. Analyse critique des choix 12
29. Limites et travaux de clôture 12
30. Conclusion 13
31. Annexes de preuve et références 14
    - 31.1. Socle pédagogique : HTML, formulaire et JSON 14
    - 31.2. Deux projets dans VS Code Remote SSH 16
    - 31.3. Sources et reproductibilité 17
    - 31.4. Planche vectorielle : cas d'utilisation 18
    - 31.5. Planche vectorielle : attribution d'un prêt 19

1. Introduction

Le projet part d'une vidéothèque dont les œuvres et les exemplaires doivent rester cohérents lorsque plusieurs utilisateurs agissent simultanément. L'enjeu n'est donc pas seulement d'afficher une liste : une copie déjà prêtée ne doit pas être attribuée une seconde fois, une collection personnelle doit rester privée et une panne ne doit pas transformer un import partiel en état métier valide.

CineVault sépare l'interface, la logique métier et la persistance selon les trois tiers du sujet. Les extensions collaboratives sont ajoutées à ce socle, tout en conservant les exercices Flask/Jinja et le fonctionnement Compose demandés. Le présent rapport relie les décisions au code, aux tests et aux campagnes datées ; la présence d'un fichier de configuration n'est jamais assimilée à une opération réussie.

2. Analyse du sujet et des cours

Le sujet RT0705TP, sur trois pages, est la référence principale. La première page demande fonctionnalités, acteurs, données et modèles d'exécution ; la deuxième précise environnement, deux projets, conteneurs, montage du code et exercices Flask/Jinja ; la troisième poursuit l'API et la présentation avec création coordonnée des conteneurs. La matrice TRACEABILITY conserve les 28 exigences du socle et distingue réalisation, vérification et écart.

Les supports C0 à C6 éclairent modélisation, sérialisation, messages, client-serveur, développement et REST. Ils servent à expliquer ce qui arrive à réception d'un message : décodage, autorisation, lecture, calcul, transaction et réponse. Docker Swarm, les comptes et les métadonnées sont des choix supplémentaires du mandat utilisateur. Leur richesse ne compense pas une absence de preuve du montage pédagogique ou d'un exercice de base.

3. Cahier des charges et critères de réussite

Le périmètre comprend catalogue, recherche et filtres, fiches, mutations versionnées, comptes et rôles, collections privées, exemplaires, prêts, réservations, import/export JSON et CSV, métadonnées, recommandations explicables et statistiques. Le stockage principal reste JSON ; une base relationnelle n'est pas substituée au travail demandé. Les deux applications restent Python/Flask et la présentation utilise Jinja, sans SPA.

Les critères de sortie sont observables : deux projets Git indépendants, échanges JSON réels, opérations protégées, persistance, Compose puis trois VM, registre accessible, CI effectivement exécutée et livrables lisibles. Les matrices du sujet et des extensions restent séparées. Un statut VERIFIED est borné à sa campagne et à ses révisions ; il ne signifie pas que toutes les évolutions ultérieures ou tous les scénarios d'exploitation sont validés.

4. Méthodologie et conservation de l'existant

La démarche suit inventaire, hypothèse, essai ciblé, correction puis revalidation. Les adresses et ressources ont été relevées dans le laboratoire ; elles n'ont pas été déduites d'une topologie théorique. Les fichiers et données préexistants ont été inspectés avant les installations ou mises à jour. Les opérations de bootstrap, création de secrets et initialisation ne sont pas des commandes à rejouer systématiquement.

Les états de reprise, commits et reçus permettent une continuation sans recommencer le projet. Les essais risquant de créer des données emploient des volumes neufs et des identités fictives ; l'exploitation conserve ses comptes et secrets. Les preuves brutes privées sont séparées des pièces publiques revues. Les doubles fichiers documentaires entièrement NUL découverts sur le poste ont été sauvegardés puis restaurés depuis leur commit ; la cause n'est pas établie et n'est pas attribuée arbitrairement à un outil.

5. Acteurs, responsabilités et charge

Les rôles fonctionnels n'ont pas tous la même représentation informatique. Les permissions API sont member, manager et admin ; le visiteur correspond à l'absence de session, tandis que l'exploitant possède des responsabilités Linux/Docker distinctes. Une connexion administrateur applicatif n'accorde donc pas l'administration des machines ni l'accès automatique à toute donnée privée.

La charge décrite dans les modèles est qualitative : lectures fréquentes du catalogue, mutations moins fréquentes, accès privés authentifiés et coût élevé du calcul de mot de passe. Aucun profil de charge issu d'entretiens ou de mesures de production n'est inventé. Le point de contention principal est le document JSON et son verrou ; le fournisseur externe introduit une autre latence, traitée par cache et délais.

Acteur	Responsabilité et limite
Visiteur	Lecture du catalogue public ; aucune mutation métier.
Membre	Ressources privées, demandes, favoris et listes.
Gestionnaire	Catalogue partagé, exemplaires, attribution et retours.
Administrateur	Comptes et rôles applicatifs ; pas d'accès automatique aux collections privées d'autrui.
Exploitant	Linux, Docker, déploiement et reprise ; rôle distinct des autorisations HTTP.

6. Personas et parcours prioritaires

Cinq personas explicitement fictifs structurent l'analyse : collectionneuse passionnée, membre occasionnel, gestionnaire, administrateur applicatif et exploitant. Leurs besoins opposent découverte rapide, classement personnel, attribution fiable, maîtrise des droits et reprise reproductible. Ils servent à vérifier les parcours ; ils ne sont pas présentés comme des personnes interrogées ni comme la preuve historique d'une recherche utilisateur.

La collectionneuse doit retrouver ses favoris et supports ; le membre doit comprendre disponibilité et attente ; le gestionnaire doit éviter une attribution conflictuelle ; l'administrateur doit préserver au moins un administrateur actif ; l'exploitant doit distinguer donnée métier, session, secret et image. Les écrans et tests sont confrontés à ces critères concrets. Une évaluation de satisfaction auprès d'utilisateurs reste une perspective, distincte de la validation technique.

7. Cas d'utilisation et autorisations

Le modèle UML décrit 14 cas principaux avec préconditions, postconditions et exceptions dans actors-use-cases.md. Le diagramme source PlantUML est conservé et son SVG a été compilé. Les liens d'héritage rendent les capacités lisibles sans assimiler les privilèges d'exploitation aux rôles du service HTTP.

Pour une mutation, les préconditions combinent session valide, rôle ou propriété, ressource existante, état métier et version attendue. Une autorisation vérifiée seulement dans le frontend serait insuffisante : l'API recommence ce contrôle sous la transaction pertinente. Les exceptions ne sont pas de simples messages graphiques ; elles déterminent le statut HTTP et l'absence ou la portée des écritures éventuelles.

Le modèle UML complet est reproduit dans la planche Fig. 8. Les préconditions et exceptions détaillées restent dans le document de modélisation associé.

8. Modèle fonctionnel

Le catalogue partagé décrit les œuvres, tandis que les copies représentent les supports disponibles. Une entrée de collection décrit le lien privé d'un utilisateur avec un film ; elle ne crée pas nécessairement un exemplaire prêté par la vidéothèque. Une demande de prêt et une réservation sont deux ressources différentes, avec leurs propres transitions.

Le découpage fonctionnel est reflété dans les blueprints, services et validateurs : comptes, catalogue, collections, circulation, transfert, métadonnées et calculs. La consultation publique ne reçoit pas les identités d'emprunteurs. Les écrans privés demandent les projections autorisées. Cette séparation évite qu'une page construite à partir d'un ensemble de données global expose accidentellement les informations d'autrui.

9. Modèle de données et exemples

Le document v2 comprend dix tableaux : users, sessions, auth_attempts, idempotency_records, movies, copies, collections, loans, reservations et audit_events. Les identifiants stables permettent les références entre œuvres, exemplaires et utilisateurs. Les rôles, personnes et genres sont représentés selon les usages réels : rôles énumérés, personnes et genres textuels ; aucune entité Review fictive n'est ajoutée pour remplir un diagramme.

Le contrat OpenAPI courant contient 42 composants de schéma, comptés directement, et le stockage 18 définitions. Le dictionnaire détaille attributs, types, contraintes et différences entre objets stockés et ressources HTTP. Les exemples publics sont synthétiques ; les validations de JSON Schema ne remplacent pas les invariants de référence, l'unicité ou les règles de transition exécutées en Python. Le manifeste des exemples doit être consulté pour la couverture exacte de la version livrée.

Le corpus exécuté couvre 42 composants OpenAPI,18 définitions de stockage et cinq schémas techniques autonomes. Les 93 échanges issus des vrais handlers couvrent les 44 couples méthode-chemin et 51 emplacements JSON. Vingt-trois tests de corruption et compatibilité vérifient le vérificateur lui-même, notamment les formats et la portabilité LF/CRLF des sources ; ils ne remplacent pas les règles métier ni les essais en réseau.

10. Modèles d'exécution et messages

Une requête est synchrone dans le chemin navigateur-présentation-API ; aucune file de messages ou exécution asynchrone métier n'est revendiquée. À réception, le serveur web vérifie notamment CSRF et formulaire, appelle l'API avec un délai borné, puis rend Jinja ou une erreur contrôlée. L'API décode le JSON, authentifie si nécessaire, valide les paramètres et exécute le traitement associé.

Dix séquences vectorielles couvrent consultation, inscription, connexion, mutation versionnée, attribution de prêt, réservation/retour, import, fournisseur, affiche et lecture privée. Elles montrent les traitements et les verrous, y compris les nuances : un GET privé peut mettre à jour last_seen et la révision ; les lectures publiques de copies et disponibilité sont séparées ; la validation de connexion Argon2 demeure sous le verrou métier. Ces modèles statiques ne sont pas des traces réseau.

La planche Fig. 9 explicite une attribution de prêt et le conflit concurrent, avec traitements à réception. Les neuf autres séquences vectorielles sont fournies dans le dépôt.

11. Architecture à trois tiers

Le navigateur est le tiers client ; Flask/Jinja constitue la présentation ; l'API Flask et son stockage forment le tiers d'accès aux données. Cette séparation existe dans les communications et les droits sur le fichier, pas seulement dans trois rectangles. Le frontend reçoit des réponses JSON puis crée le HTML ; il ne lit pas directement le fichier du catalogue.

Le manager Swarm appartient au contrôle d'infrastructure. Il ne remplace aucun de ces trois rôles logiques et n'héberge pas de tâche applicative dans le mode Drain observé. Le découpage permet de tester les clients HTTP et les handlers séparément, puis de vérifier de vrais échanges entre conteneurs et entre machines.

La figure Fig. 1 précise la responsabilité exclusive de l'API sur le fichier métier.

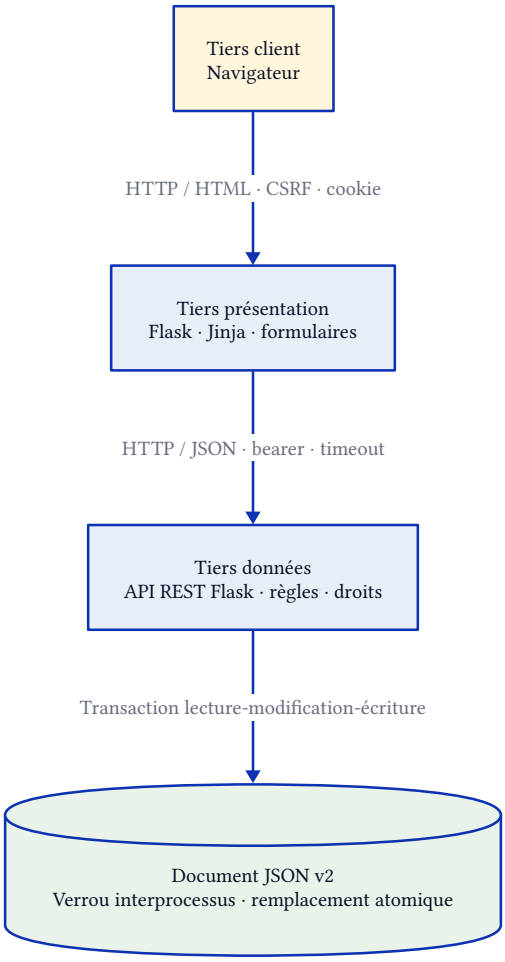


Fig. 1. – Trois tiers logiques. Le manager assure l'orchestration, hors du chemin métier.

12. Inventaire du frontend et des scripts

L'inventaire documentaire de la présentation rattache chaque page aux données reçues, appels API, traitements et erreurs. Le socle relevé comporte 36 règles URL et 28 templates. Le correctif de frontière ajoute response_contracts.py aux modules existants ; les scripts de qualité et de publication sont distingués des routes utilisateur. Les nombres ne doivent pas être réutilisés après une modification sans nouveau relevé.

Les parcours incluent accueil, catalogue, fiche, comptes, profil, administration, collections, demandes et gestion des prêts, réservations, transfert, métadonnées et statistiques. Les templates partagent navigation, formulaires et affiches avec crédits. Les états vides, refus d'accès, conflit de version et indisponibilité font partie des comportements attendus. Un inventaire statique n'est pas une preuve que chaque état a été inspecté dans Chrome.

13. Spécification REST

Le contrat versionné /api/v1 distingue ressources et transitions. Les succès utilisent notamment 200,201 et 204 ; les erreurs 422,401,403,404,409,412 et 428 expriment validation, accès, absence, conflit et condition de version. Le statut 429 porte une limitation ;503 indique une indisponibilité ou une réponse du service non exploitable. La forme exacte et les champs sont décrits dans OpenAPI et les tests de contrat.

L'ETag est dérivé de la représentation persistée et doit être fourni dans If-Match pour les opérations concernées. L'idempotence d'import possède sa propre clé et son reçu. Ces mécanismes répondent à deux problèmes distincts : éviter une écriture sur une version dépassée et éviter de réappliquer un même lot après incertitude. Le client ne rejoue pas automatiquement une mutation simplement parce que le transport a échoué.

Famille /api/v1	Méthodes	Réception et réponse
movies / copies	GET, POST, PUT, DELETE	Lecture publique ; mutations manager/admin, validation et If-Match.

Famille /api/v1	Méthodes	Réception et réponse
auth / me / users	POST, GET, PATCH, PUT	Session, profil et contrôle des rôles ; secrets jamais renvoyés.
collections	GET, POST, PUT, DELETE	Propriété de la ressource vérifiée côté API.
loans / reservations	GET, POST	Transitions métier, conflits, priorité et version.
catalogue/import, export	POST, GET	Porteur JSON, validation du lot et idempotence.
metadata / posters	GET	Cache, fournisseur et affiches sous conditions.
statistics / recommendations	GET	Calculs bornés aux données et autorisations présentes.

14. Infrastructure RemoteLabz

Trois VM Ubuntu 24.04.5 ont été installées et inventoriées. Chacune dispose de deux CPU et d'environ 1966 Mio de mémoire ; Docker 29.9.3, Compose 5.7.0 et Git 2.43.0 ont été constatés. Les clés hôtes SSH ont été comparées aux consoles avant utilisation en politique stricte. Le poste Windows sert au développement, à l'administration, aux captures et à la construction documentaire.

L'accès courant utilise les connexions SSH locales, après rétablissement du VPN déjà configuré. La session du navigateur intégré n'a pas été assimilée à une session Chrome distincte. Le laboratoire n'a pas été quitté ni réinitialisé. L'ordre de boot autonome des VM 2/3 reste une réserve : leur fonctionnement actuel est démontré, mais un redémarrage aveugle ne prouverait pas une reprise sûre.

La figure Fig. 2 distingue le trafic métier du contrôle d'orchestration.

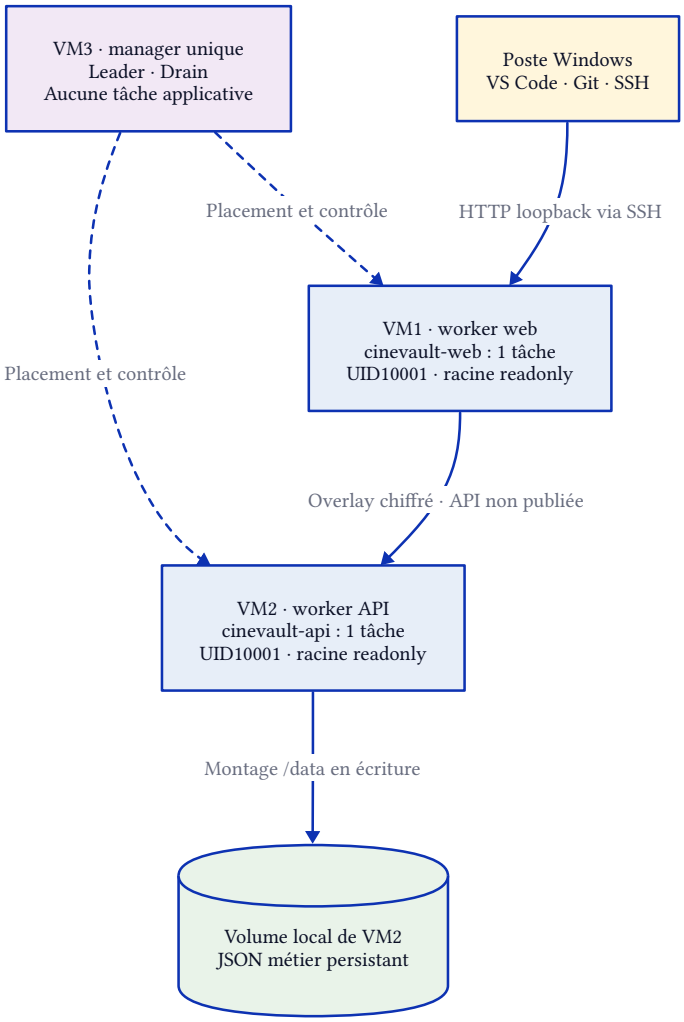


Fig. 2. – Placement effectif sur trois VM. Les adresses du laboratoire restent dans les preuves privées.

15. Docker Compose et volume partagé

Compose constitue le premier environnement de coordination, conformément au protocole du projet. Le code de chaque application est monté depuis l'hôte en lecture seule ; le stockage est fourni par un volume séparé en écriture. Cette démonstration pédagogique est conservée même si le déploiement Swarm utilise des images immuables.

Une nouvelle pile isolée a été vérifiée sur VM 1 aux révisions API 6a86f2e et web 4883876. Seize contrôles ont confirmé montages effectifs, empreintes du code hôte/conteneur, visibilité d'un fichier témoin, services sains, exercices A/B/C et conservation de la tâche Swarm. Les témoins seuls ont été retirés et les clones sont propres. Le volume de test est distinct des données Enterprise ; la campagne ne démontre pas un rechargement automatique du code.

16. Docker Swarm et distribution des images

Le cluster possède un manager Leader en Drain et deux workers. Les contraintes de placement assignent web et API aux machines prévues. Les services utilisent un Overlay chiffré ; la compatibilité réseau a été contrôlée par mesures et échanges effectifs, au-delà d'une simple déclaration de ports. Une seule réplique API monte le volume local de son worker.

La distribution initiale par préchargement SFTP a été remplacée par GHCR public. Chaque digest déployé est relié au manifeste brut du registre, à son config.digest et à l'image scannée par la CI. Les téléchargements anonymes ont été exécutés sur les trois nœuds avec configuration Docker vide. L'ID présenté par les moteurs CI et laboratoire peut désigner des niveaux différents ; la chaîne de manifeste évite une comparaison trompeuse.

17. Développement du frontend et robustesse

La présentation conserve Flask et Jinja. Le client HTTP central gère délais, absence de redirection, statuts d'erreur, bearer et signatures de contexte. Les erreurs de mutation conservent les champs et avertissent de vérifier le résultat avant de réessayer. Les clients API injectés dans les tests restent un mécanisme explicite d'isolation, distinct d'un essai interservices.

Une réponse JSON décodable mais de mauvaise forme provoquait des erreurs 500 : films:null, film sans titre, pagination nulle ou création vide. Les contrôles structurels ont été placés dans le client natif avant adaptation et rendu. La revue indépendante a ajouté deux cas :204 inattendu et surrogate Unicode non encodable. Ils sont devenus des 503 contrôlés, tandis que catalogues vides, champs métadonnées nuls, accents, emoji et affiches facultatives restent compatibles. Soixante-quatorze cas supplémentaires portent la suite web à 159 tests.

Le parcours Tab réel du catalogue atteint ses 14 contrôles attendus. Recherche, état vide et retour au catalogue ont été exercés à 390 et 820 pixels sans débordement de page. Les placeholders mesurés à 3,56:1 ont été corrigés avec le jeton de texte secondaire existant et une opacité explicite ; après déploiement, le contraste observé atteint 8,99:1. La console du parcours ne contient aucune erreur ni avertissement. Ce contrôle ciblé ne vaut pas certification WCAG de tous les écrans.

```
if response.status_code == 204:
    if allows_no_content(method, path):
        return None
    raise APIError(503, "Réponse invalide du service de données.")
try:
    result = response.json()
except ValueError as exc:
    raise APIError(503, "Réponse invalide du service de données.") from exc
if not valid_response(method, path, result):
    raise APIError(503, "Réponse invalide du service de données.")
return result
```

Extrait de la frontière HTTP native au commit 10612c9. Le contrôle précède tout accès aux champs et toute adaptation destinée à Jinja.

18. Développement du backend

Les routes API sont séparées des validateurs, services métier, autorisations et persistance. Les opérations qui peuvent attendre un fournisseur externe réalisent des contrôles avant le réseau puis revalident l'état pertinent au commit. La transaction ne doit pas être étendue à une attente réseau sans nécessité, sous peine de bloquer toutes les autres écritures.

Les ressources publiques sont projetées sans données privées ; les accès authentifiés relisent la session et le rôle. Les tests couvrent également les refus et les conflits, car une opération incorrectement refusée ou acceptée est un comportement métier à part entière. Les journaux publics de CI et les fichiers OpenAPI rendent le code vérifiable, sans exposer le magasin du laboratoire.

19. Authentification et autorisations

Les mots de passe sont hachés avec Argon2id. Les sessions utilisent des jetons opaques dont seule l'empreinte est conservée côté API ; expiration, invalidation et changements de droits sont contrôlés. Les cookies et CSRF protègent les formulaires, mais ne remplacent pas le contrôle d'autorisation côté API. Le premier administrateur est créé par une opération locale explicite, sans mot de passe par défaut publié.

L'inscription réserve un budget avant le hachage :20 calculs par source et 60 globalement par fenêtre de 60 secondes, avec verrou d'admission distinct et capacité métier recontrôlée au commit. Le frontend signe le pair observé, la méthode, le chemin, l'horodatage et l'empreinte du corps exact avec une troisième clé dédiée. Un X-Forwarded-For libre n'est pas utilisé comme identité. Ces protections bornent un scénario précis ; elles ne constituent pas une immunité générale au déni de service.

20. Collections et exemplaires

Un film représente l'œuvre ; une copie possède son propre identifiant, support, emplacement et état. La disponibilité est dérivée des prêts et de la file de réservation. Une entrée de collection relie un utilisateur à un film selon un type : possédé, favori, vu, à voir ou liste personnalisée. Les supports possédés d'un membre ne sont pas automatiquement les copies prêtables de la vidéothèque.

L'API protège la propriété des entrées même contre un administrateur applicatif demandant une collection d'autrui. Les exemplaires relèvent des fonctions de gestion. Les mises à jour versionnées et les règles de suppression empêchent de supprimer silencieusement les références encore utilisées par l'historique. Les tests d'isolation et les parcours réels complètent les contrôles de schéma.

21. Emprunts et réservations

Une demande de prêt commence sans exemplaire attribué. Le gestionnaire consulte des options qui ne réservent pas à elles seules une copie ; l'attribution effective recontrôle disponibilité, demande, version et priorité dans la transaction. L'exemplaire ne peut appartenir à deux prêts actifs. Le retour conserve l'historique et ne lance pas automatiquement un prêt au suivant.

La réservation représente une attente ordonnée avec expiration opportuniste. Les statuts waiting, fulfilled, cancelled et expired sont distincts ; la priorité est recalculée à partir de l'état réel. Les essais concurrents ont montré un succès et un conflit lors d'une double attribution. Les exemples de cycle de vie permettent aussi de lire tous les états sans les confondre avec des données de production.

22. Persistance JSON et reprise

Le verrou fichier couvre lecture, modification et écriture entre processus. Le document candidat est entièrement validé avant remplacement ; un fichier corrompu n'est pas remplacé par une initialisation vide implicite. Un remplacement atomique protège la structure, tandis que synchronisation et gestion des erreurs traitent la durabilité dans la mesure fournie par le système. Une erreur après remplacement peut rendre le résultat d'une mutation incertain.

Un instantané cohérent n'est pas à restaurer aveuglément avec ses anciens accès. La procédure de reprise Enterprise crée une cible absente, neutralise mots de passe et sessions restaurés, conserve les prêts et prévoit de nouveaux accès vérifiés. Les essais locaux sur cibles isolées ne prouvent pas une reprise après perte physique de disque. La mise à jour courante, elle, préserve volontairement comptes et sessions ; ce n'est pas la même opération.

23. Extensions externes et calculs

Les métadonnées Wikimedia sont opt-in, avec User-Agent public, cache partagé, quotas réservés avant HTTP et fonctionnement dégradé. La stack distante a réellement réussi recherche Metropolis et aperçu Q151599, avec Fritz Lang et conditions de média. Cet aperçu n'a pas créé de film. La proposition doit être contrôlée avant enregistrement ; provenance d'une entité et droits d'une affiche ne garantissent pas la justesse de chaque champ futur.

Les statistiques sont calculées à partir des ressources présentes et des permissions. Les recommandations sont déterministes et accompagnées de raisons et points ; elles ne sont pas présentées comme du machine learning. Leur cohérence est testable, mais une pertinence algorithmique auprès d'utilisateurs ne peut pas être déduite d'un résultat HTTP 200.

24. Stratégie de tests et résultats

Les suites unitaires et de contrat isolent règles, stockage, droits, réponses et concurrence. Les essais Compose exercent réellement HTTP et les volumes ; les campagnes distribuées contrôlent nœuds, placement, réseau, images et parcours. Les visites Chrome et captures sont des observations de l'interface, distinctes des assertions in-process. Chaque couche apporte une preuve différente.

Les CI API 42536728355 et web 42640090796 ont été exécutées et leurs journaux de tests lus. Les jobs qualité Windows/Ubuntu et construction-santé-scan-publication ont réussi. Dans ces événements de branche par défaut, le job d'image sans publication est intentionnellement ignoré, car mutuellement exclusif. Les résultats historiques locaux et récents distants restent datés ; les nombres ne sont pas additionnés pour fabriquer un taux de couverture global.

Campagne	Résultat	Périmètre
API 6a86f2e	173 / 173	Windows / Ubuntu
Web f3d10cb	159 / 159	Windows / Ubuntu
Compose auth historique	171 contrôles	API 4b15406 / web 9b1da4f

Campagne	Résultat	Périmètre
Compose pédagogique neuf	16 contrôles	API 6a86f2e / web 4883876
Mise à jour Swarm courante	25 contrôles	API 6a86f2e / web f3d10cb

25. Sécurité et limites du contrôle

La sécurité associe vérification des droits, sessions et CSRF, validation des entrées et réponses, secrets séparés, images non privilégiées, racine readonly et capacités retirées. Trois secrets Docker stables sont montés uniquement aux services concernés. Les profils VPN, clés privées, mots de passe et magasins métier sont exclus des dépôts, preuves publiques et rapport.

L'audit source a identifié un coût d'inscription non borné et un regroupement indésirable des sources de connexion derrière le frontend ; admission et contexte signé les corrigent dans leur périmètre. Le scan des images Grype 0.121.2 ne trouve aucun High/Critical mais conserve cinq Medium et un Low par image. Il ne certifie pas l'absence de vulnérabilité ; HTTP sans HTTPS public et la connexion Argon2 sous verrou métier restent des limites explicites.

26. Résultats observés

Le 9novembre à 16 h 04 Paris, trois nœuds sont Ready, le manager est Leader/Drain et les deux services sont sains sur leurs workers. La version web f3d10cb est exécutée depuis son digest GHCR vérifié ; l'API 6a86f2e et son conteneur sont restés inchangés durant cette mise à jour. Le code de contrôle des réponses dans le conteneur correspond par SHA-256 au commit publié.

Les 25 contrôles ont confirmé données métier et comptes identiques, secrets inchangés, session ouverte avant déploiement encore utilisable et nouvelle connexion signée fonctionnelle. La comparaison exclut sessions, compteurs et révision, qui sont opérationnels. La présence d'un instantané préalable et la conservation de l'état ne signifient pas qu'une restauration après panne a eu lieu.

27. Problèmes rencontrés et corrections

Le montage tmpfs court avait été accepté par la validation de stack mais ignoré dans le service effectif du laboratoire : Gunicorn ne disposait alors pas d'un dossier temporaire inscriptible. Le passage à la forme longue de montage a été vérifié dans les Mounts, puis par la santé effective. Cet incident illustre la différence entre YAML accepté et runtime correct.

La première CI Linux d'image échouait car l'archive Docker 0600 était illisible par le scanner UID 65532. Seul ce fichier reçoit désormais 0444, dans un dossier privé ; le scanner demeure nonroot et le montage readonly. Les tests rouges puis verts et les CI suivantes attestent la correction. Les erreurs JSON frontend ont été traitées de la même manière : reproduction, garde à la frontière, revue indépendante, nouvelle CI et nouveau déploiement.

28. Analyse critique des choix

JSON répond au sujet et reste lisible, transportable et testable. En contrepartie, le document entier et son verrou bornent la concurrence et la capacité ; la séparation en microservices ne supprime pas ce goulot. Un SGBD transactionnel serait une alternative pour une autre charge, mais le substituer ici aurait retiré la contrainte pédagogique. Une API unique sur le volume propriétaire est donc un choix cohérent, avec une indisponibilité assumée lors d'une panne de ce nœud.

Swarm apporte placement et mises à jour, sans rendre le manager unique ni le volume local hautement disponibles. Les images immuables et digests rendent la version déployée identifiable ; ils ne garantissent pas des builds bit-identiques à tout instant lorsque certains paquets système sont résolus au build. Les dépendances Python et bases sont verrouillées, et les images réellement construites sont contrôlées : cette provenance mesurée est la propriété revendiquée.

29. Limites et travaux de clôture

Le laboratoire n'offre pas de preuve de haute disponibilité, de HTTPS public, de bascule de disque ni de boot autonome VM 2/3. Les parcours Chrome vérifiés sont ciblés ; ils ne constituent pas une certification exhaustive d'accessibilité. Les personas sont fictifs et l'évaluation de pertinence des recommandations reste technique. Ces limites ne sont pas masquées par les tests réussis.

Les matrices associent les exigences aux sources, campagnes et pièces datées ; les exemples sont reliés à leurs schémas. Les contrôles du PDF, la reproduction et l'audit indépendant de livraison sont consignés séparément dans le dossier documentaire. Les limites techniques restent explicites dans cette édition, sans transformer une absence de haute disponibilité ou de HTTPS public en réussite supposée.

30. Conclusion

CineVault réunit une présentation Flask/Jinja, une API REST et un stockage JSON transactionnel dans une architecture effectivement répartie. Le projet conserve le socle pédagogique et démontre Compose avant Swarm, deux projets Git, un montage de code distinct du volume métier, des CI réelles et des images de registre identifiables.

La valeur de cette réalisation repose sur la traçabilité : les états annoncés sont rattachés à des campagnes, les corrections à des défauts reproduits et les limites à des preuves manquantes ou à des choix explicites. La livraison finale doit conserver cette précision. Un résultat vérifiable est plus utile qu'une affirmation globale de conformité, de sécurité ou de résilience que les observations ne suffiraient pas à établir.

31. Annexes de preuve et références

31.1. Socle pédagogique : HTML, formulaire et JSON

Les trois captures suivantes proviennent du parcours Chrome du 9 novembre dans la pile Compose isolée. Le film et ses personnes sont fictifs. La comparaison fichier JSON, réponse API et valeurs Jinja est conservée séparément ; une capture de formulaire seule ne démontrerait pas le traitement.

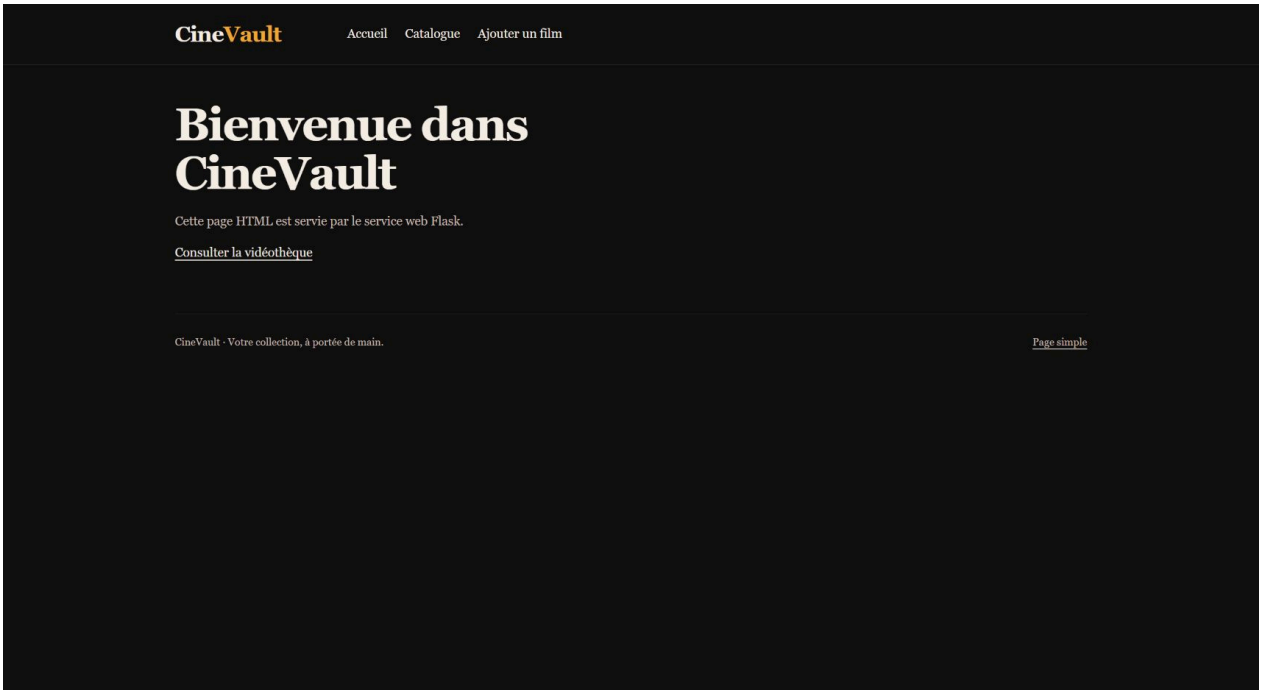


Fig. 3. – Exercice A : page HTML réellement servie par Flask.

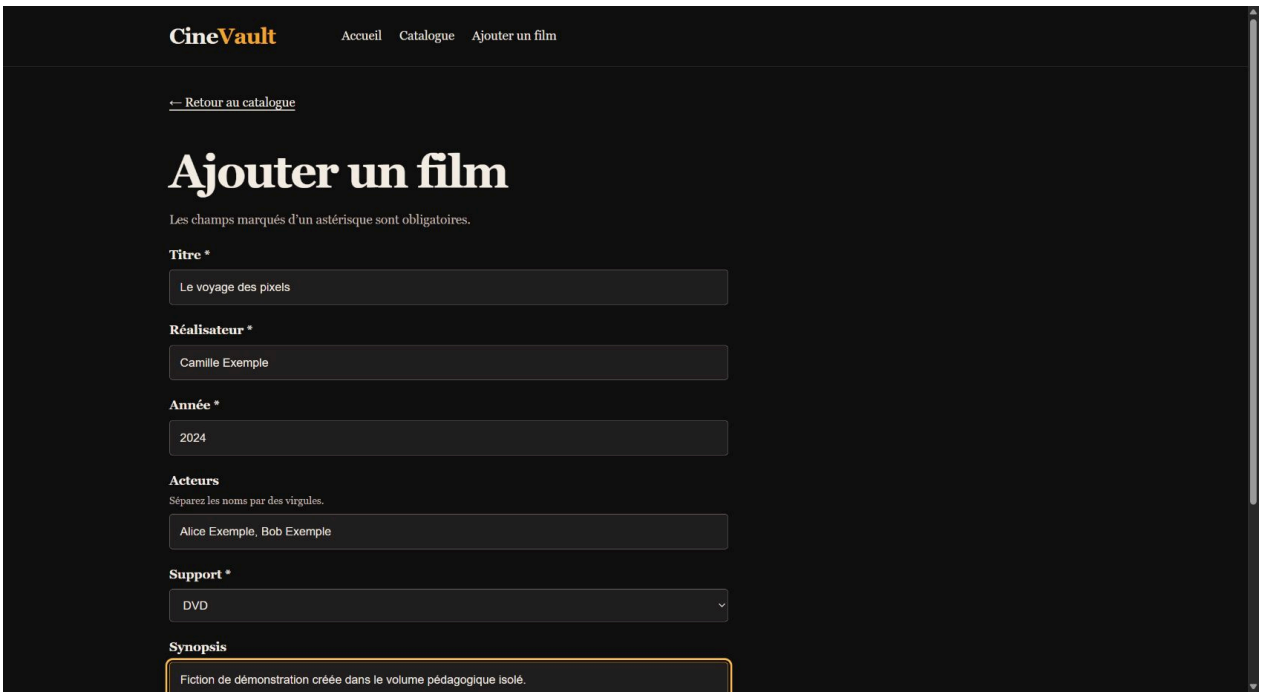


Fig. 4. – Exercice B : saisie du formulaire avant soumission.

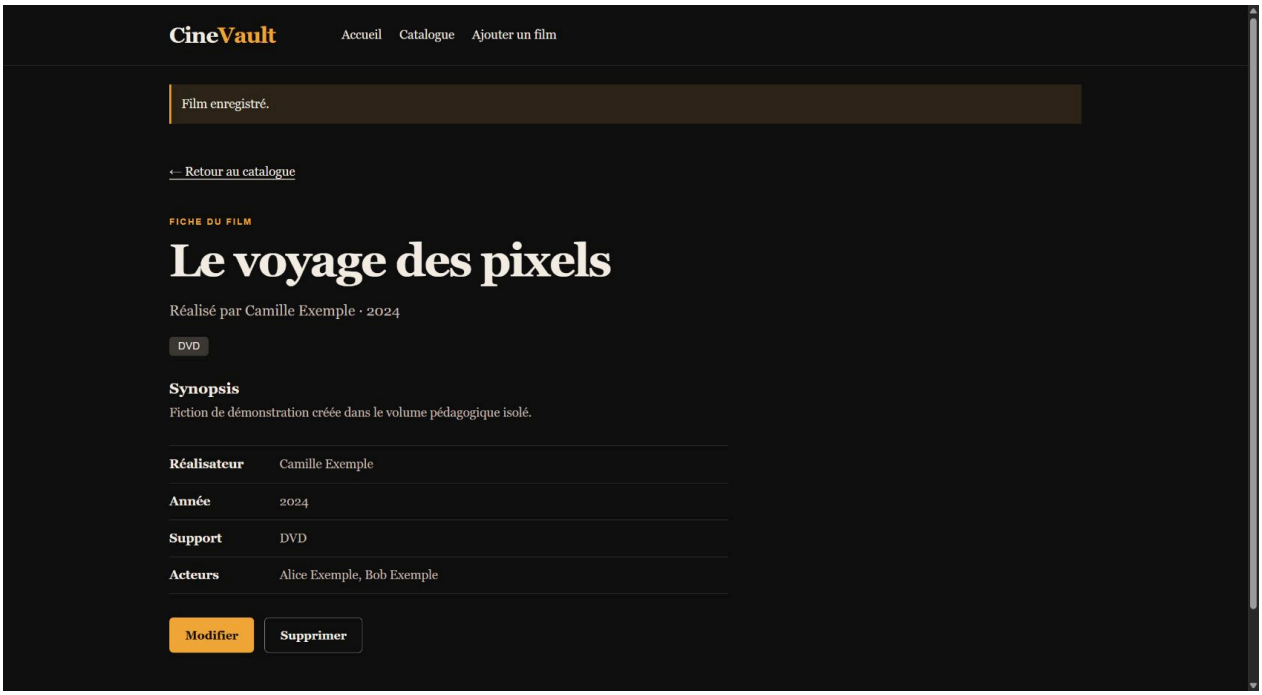


Fig. 5. – Exercice C : confirmation et fiche Jinja après le POST réel.

31.2. Deux projets dans VS Code Remote SSH

La connexion et l'ouverture effective des deux projets sont visibles. L'authentification par clé est attestée par un diagnostic OpenSSH expurgé séparé ; aucune clé privée ni journal brut n'est publié.

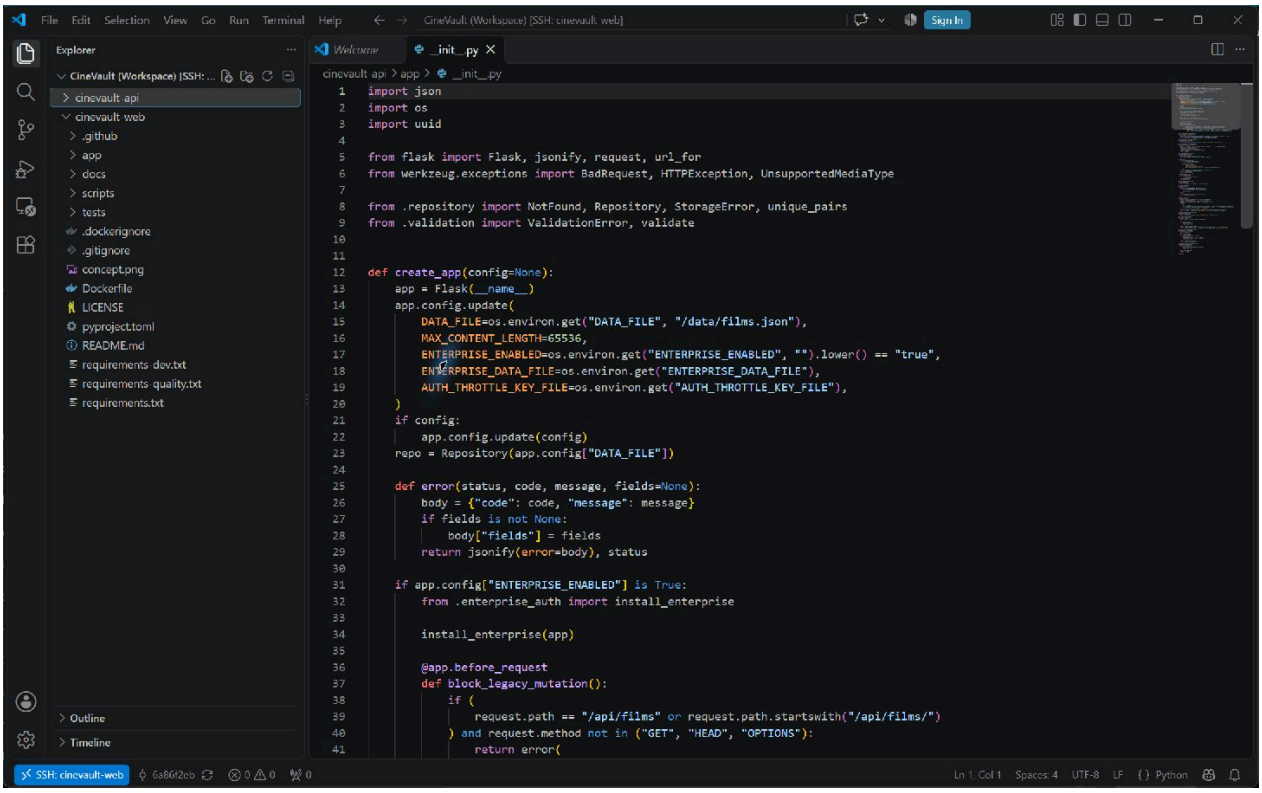


Fig. 6. – Projet API ouvert sur le worker web dans l'espace de développement séparé.

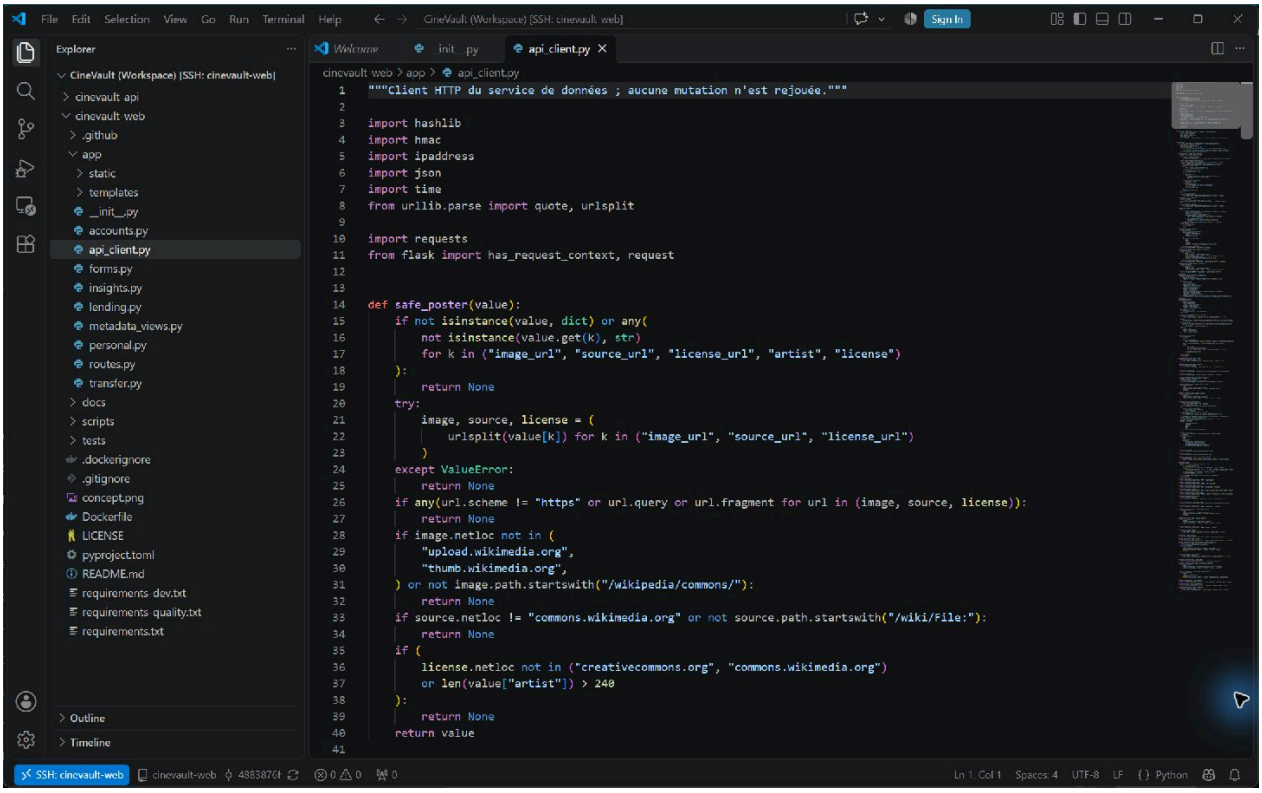


Fig. 7. – Projet de présentation et client HTTP ouverts dans le second dépôt Git.

31.3. Sources et reproductibilité

Les sources publiques et leurs historiques sont disponibles dans les trois dépôts suivants. Les fichiers README, les matrices d'exigences et les manifestes précisent les limites temporelles et la portée des preuves.

- API et stockage JSON.
- Présentation Flask/Jinja.
- Infrastructure, modèles, preuves et rapport.
- CI API 173 tests par plateforme.
- CI web 159 tests par plateforme.

Les supports C0 à C6 et le sujet RT0705TP constituent les sources pédagogiques remises pour la mission ; ils ne sont pas redistribués. Les choix de formatage du présent document utilisent la documentation officielle Typst et le compilateur portable 0.15.1, dont l'archive a été vérifiée contre le digest officiel.

31.4. Planche vectorielle : cas d’utilisation

Cette planche A2 conserve les quatorze cas, les acteurs et les notes du modèle complet, à une taille lisible dans son format original. Elle peut être agrandie sans pixellisation. Les liens représentent des capacités fonctionnelles, pas une trace réseau.

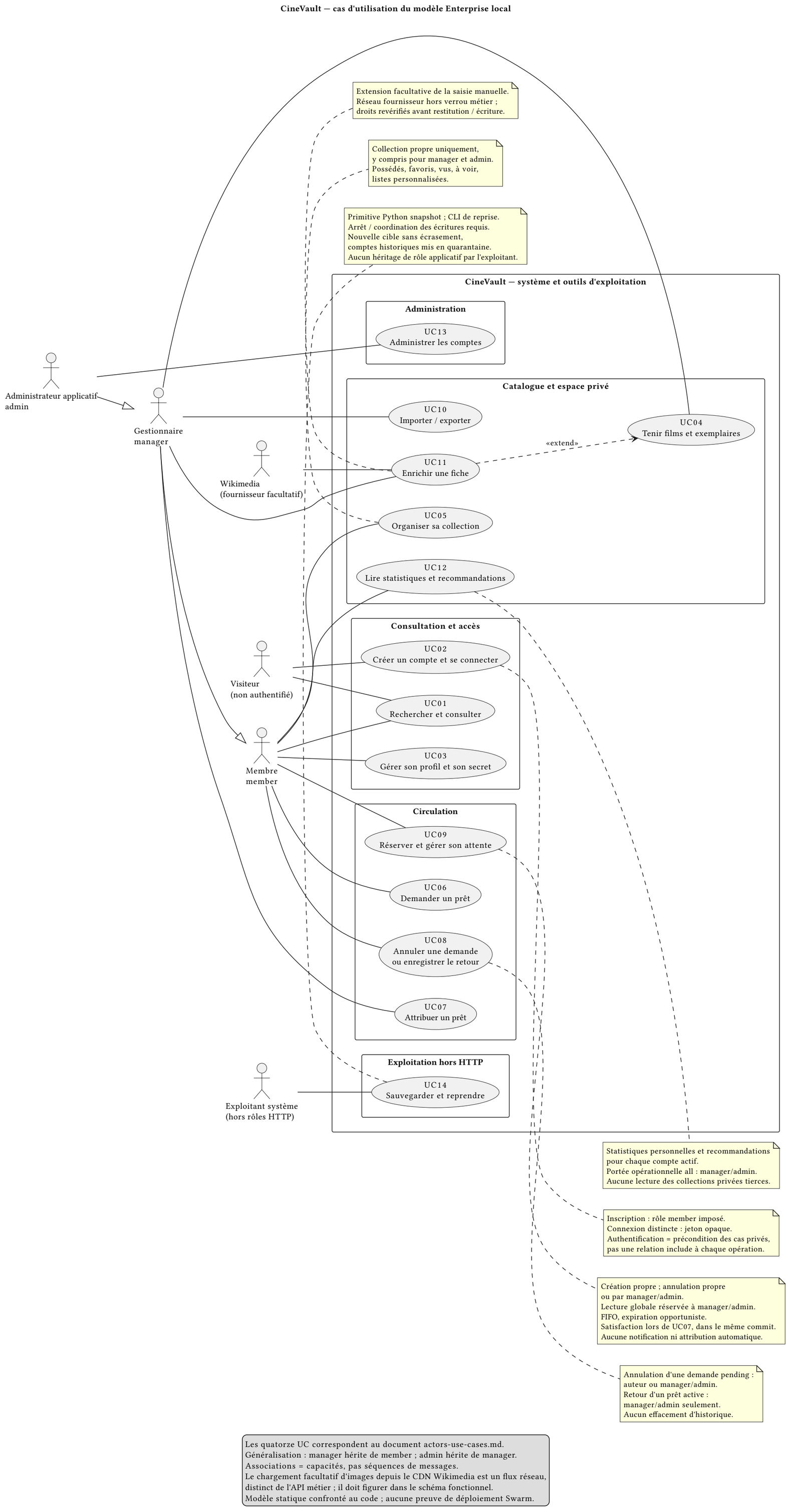


Fig. 8. – Diagramme UML UC01 à UC14, rendu depuis le PlantUML versionné et vérifié statiquement.

31.5. Planche vectorielle : attribution d’un prêt

La consultation des options ne réserve pas une copie. L’attribution, la priorité et la satisfaction de réservation sont recontrôlées dans une seule transaction. Le bloc alternatif décrit un second message concurrent ; il ne simule pas une capture d’exécution.

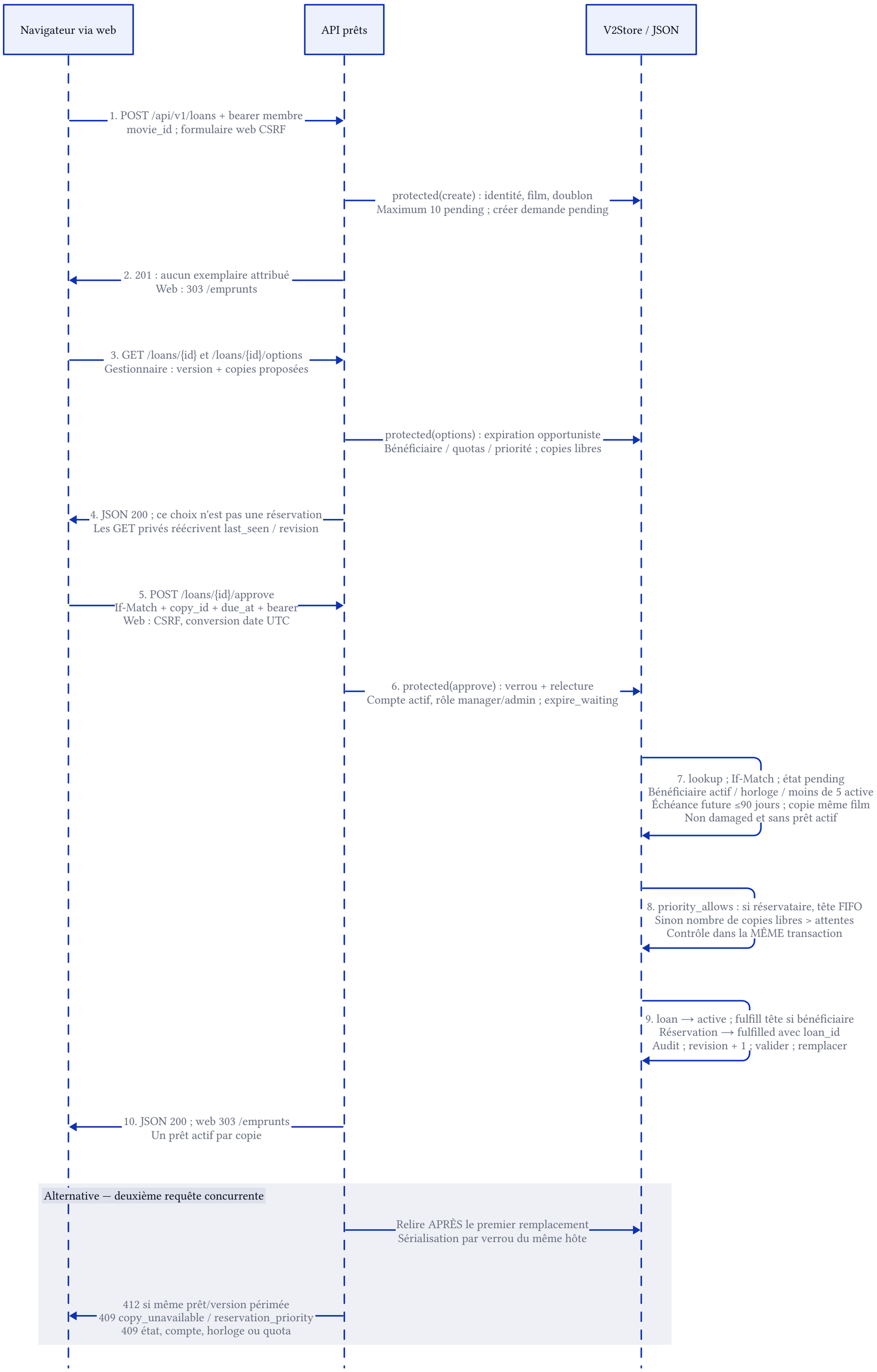


Fig. 9. – Séquence UC06 et UC07 : demande, options, attribution et conflit, avec validation et remplacement du document JSON.